Advanced Object Oriented Programming Case Study

Section: 5

Guide: Dr. Rudra KrishnaSrija

Team Number: 16

Team Members And Roll Numbers:

SAI SURENDRANATH PENDYALA - 2520030157

BURGULA RISHIK - 2520030314

DHANUSH - 2520030014

Case Study Number And Title:

Case Study - 5: User Accounts , Email Verification, And Contact Details

About Casestudy:

1. Problem Statement:

A Software-as-a-Service (SaaS) company wants to develop a User Account Management System that supports user registration, email verification, optional phone numbers, postal addresses, account activation, suspension and deletion. However, the existing application faces several problems, such as users registering with incorrectly formatted email addresses, users being marked as verified without recording a verification date, and verification tokens being accepted even after their expiry dates. Phone numbers are stored in different formats, and some postal addresses contain incomplete information. Additionally,

suspended users are still able to log in, and multiple accounts are being created using the same email address with different capitalization. These problems indicate poor data validation, improper handling of account states and a lack of strong domain modelling principles. Therefore, the system needs to be redesigned using proper validation, encapsulation, entities and value objects to ensure data consistency, security and correctness.

2. Problem Analysis:

A SaaS company wants to develop a User Account Management System that supports user registration, email verification, optional phone numbers, postal addresses, account activation, suspension and deletion. However, the existing system has several design problems, including users registering with invalid email addresses, users being marked as verified without recording the verification date, and expired verification tokens being accepted. Phone numbers are stored in different formats, and postal addresses contain incomplete information. Additionally, suspended users can still log in, and duplicate accounts are created using the same email address with different capitalization. These issues lead to data inconsistency, security risks and incorrect account management.

3. Invariant Identification:

An invariant is a business rule or condition that must always remain true in a system. In the User Account Management System, several invariants must be maintained to ensure data consistency, security and correctness. Every email address must follow a valid format to prevent incorrect registrations. If an email is marked as verified, its verification date must also be recorded. Verification tokens must be valid and should not be accepted after their expiry dates. Phone numbers must be stored in a consistent format to avoid different representations of the same number. Postal addresses must contain all required information, such as street, city, postal code and country, to ensure completeness. Only active and verified user accounts should be allowed to log in, while suspended and deleted accounts must be prevented from accessing the system. Finally, every email address must be unique across all registered users, regardless of uppercase or lowercase letters, to prevent duplicate accounts. These invariants help maintain reliable user information, prevent invalid states and protect the application from security issues.

4. Domain Classification:

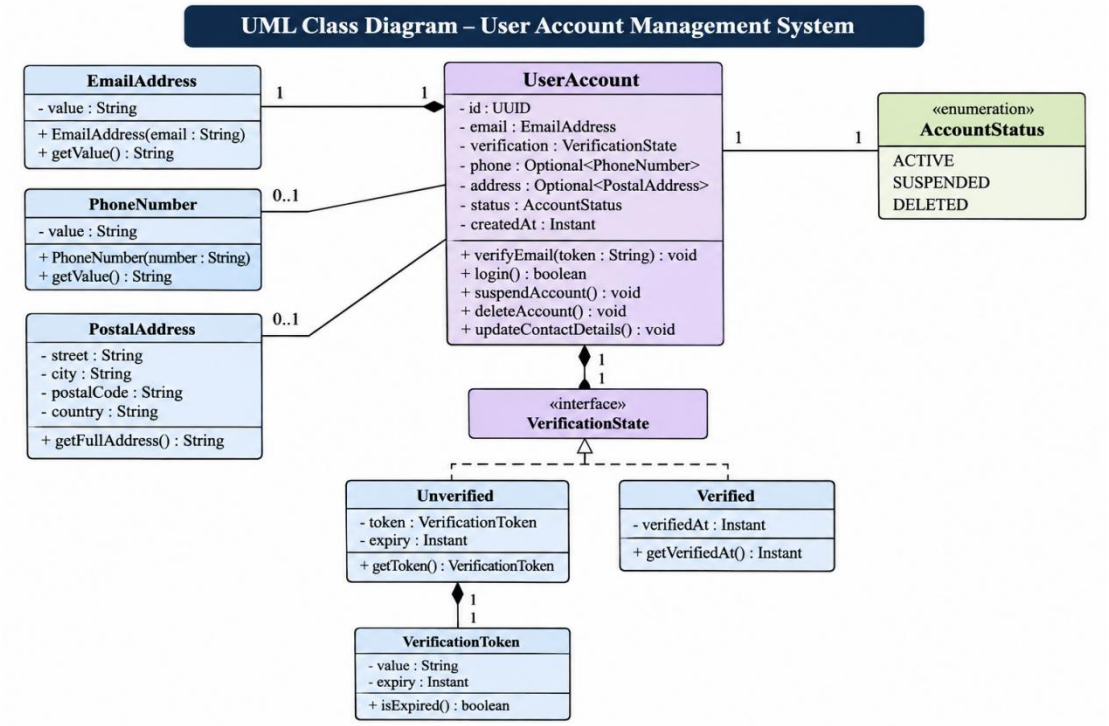
Domain classification divides the system objects into Entities and Value Objects based on their identity, equality and lifecycle. In the User Account Management System, User Account is classified as an Entity because it has a unique User ID that remains the same even when its email address, phone number or postal address changes. User Identifier, Email Address, Phone Number, Postal Address and Verification Token are classified as Value Objects because they are identified by their values rather than having separate identities. A User Identifier represents the

unique identity of a user, an Email Address is identified by its normalized email value, and a Phone Number is identified by its properly formatted number. A Postal Address is identified by its street, city, postal code and country details, while a Verification Token is identified by its token value and expiry information. Value Objects should be immutable and validated when they are created to prevent invalid data and maintain consistency.

5. Design Justification:

The proposed design uses strong domain modelling principles and object-oriented programming concepts to prevent invalid states and protect business rules. Encapsulation protects internal data from unauthorized modifications, while immutability ensures that Value Objects cannot be changed after creation. Domain-specific classes such as Email Address, Phone Number and Postal Address perform validation and prevent invalid information from being stored. A sealed interface is used to represent email verification states, preventing contradictory states such as an email being verified without a verification date. Enums are used to restrict account status to predefined values such as ACTIVE, SUSPENDED and DELETED. Optional is used to represent phone numbers and postal addresses that may or may not be provided. Verification tokens are checked against their expiry dates, and only active and verified users are allowed to log in. A database unique constraint ensures that duplicate email addresses cannot be registered, even during simultaneous registration requests. Overall, this design improves data consistency, security, reliability, maintainability and prevents invalid states in the application.

6. UML Class Diagram



****END OF DOCUMENT****